

[Introductory] Integration with Ballerina

Training Objective

The goal is to teach integration-centric Ballerina features. Once completed, you will be able to get a look and feel of Ballerina and get started with performing integrations with Ballerina.

Areas Covered

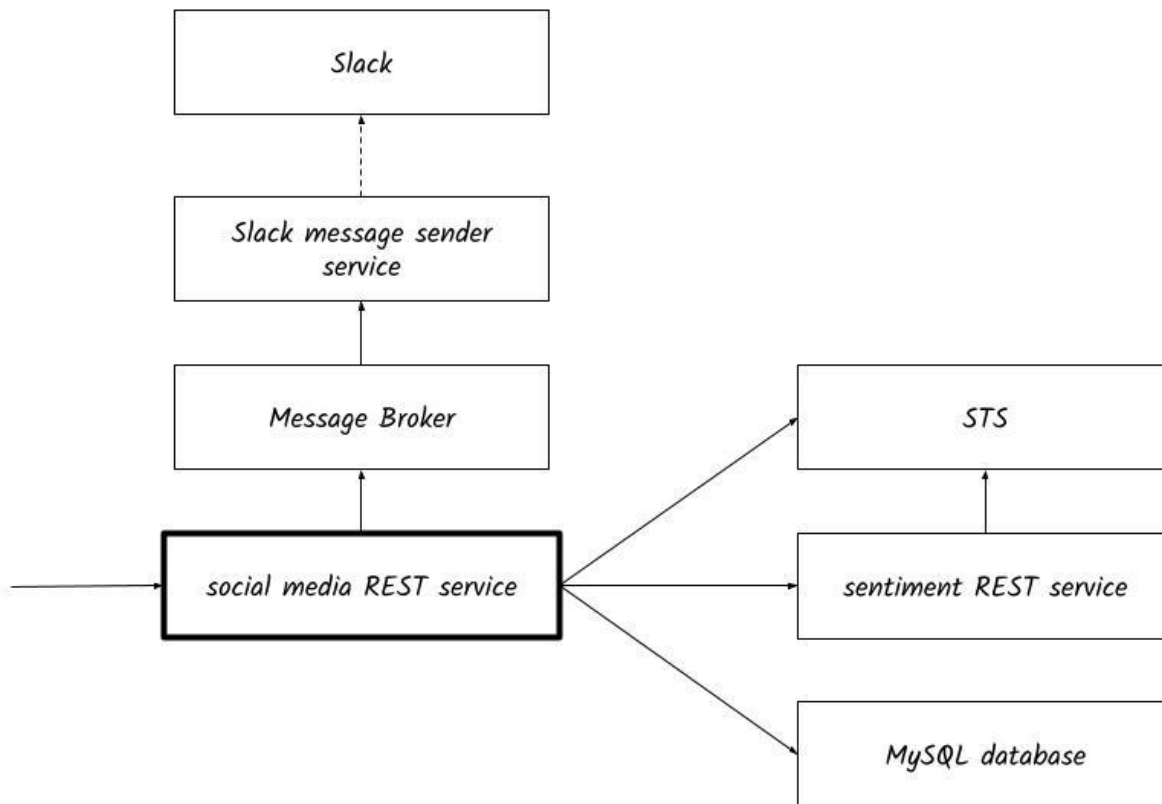
1. Writing REST APIs with verbs, URLs, data binding, and status codes
2. Accessing databases
 - a. Using JDBC or DB connector
 - b. Transactions
 - i. Rollback and Commit
3. Configurability
4. Data transformation with the data mapper
5. HTTP client
6. Resiliency
 - a. Retry
7. Writing tests
8. Using connectors
 - a. Slack
9. OpenAPI specification, client stubs, and central
10. Adding validations
11. Security
 - a. OAuth2
12. Error handling
13. Ballerina concurrency
14. Integrating a message broker

Prerequisites

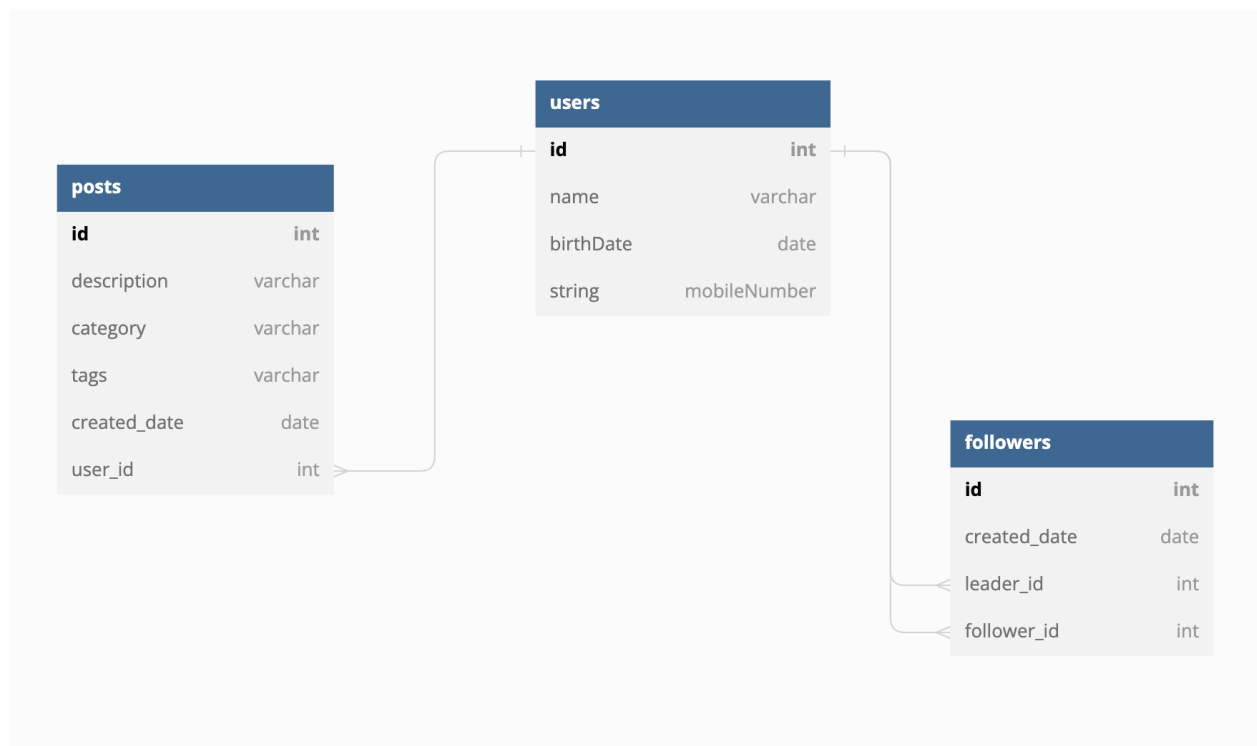
1. Install the [latest Ballerina version](#)
2. Set up VS Code by installing the Ballerina extension

Scenario

The scenario is based on a simple API written for a social media site (like Twitter), which has users, associated posts, and followers. Following is the high-level component diagram.



During the training session, you will be developing the social-media REST service step by step. This service includes a list of REST resource endpoints. While implementing each resource, you will have to connect to different endpoints as needed. To write the initial service, you will be connecting to a MySQL database. Following is the entity relationship diagram.



Then, you will connect to the sentiment analysis endpoint to validate the sentiment of each post. If the sentiment is negative, then, the posts will be rejected. As an extension, you will secure the service with OAuth2 and update the client to connect to the secured endpoint.

Toward the end of the training session, you will be sending a notification to a Slack channel whenever that particular user posts something. This feature, which shows you how to perform event-driven integration, is implemented using Slack and NATS.

Following is the service description for the social media service.

```

Unset
type SocialMedia service object {
    *http:Service;

    // users resource
    resource function get users() returns User[]|error;
    resource function get users/[int id]() returns
    User|UserNotFound|error;
    resource function post users(@http:Payload NewUser newUser) returns
    http:Created|error;
  }
  
```

```

    resource function delete users/[int id]() returns
    http:NoContent|error;

    // posts resource
    resource function get users/[int id]/posts() returns
    PostMeta[]|UserNotFound|error;
    resource function post users/[int id]/posts(@http:Payload NewPost
    newPost) returns http:Created|UserNotFound|PostForbidden|error;
};

```

Set up the environment

- For details on setting up the environment, see [Setup environment](#).

Sample code and resources

- <https://github.com/ballerina-guides/social-media-demo>

Additional learning materials

- <https://ballerina.io/learn/by-example/http-service-data-binding/>
- <https://ballerina.io/learn/by-example/http-client-data-binding/>
- <https://ballerina.io/learn/by-example/configurable-variables/>
- <https://ballerina.io/learn/by-example/mysql-query-row-operation/>
- <https://ballerina.io/learn/by-example/mysql-query-column-mapping/>
- <https://ballerina.io/learn/by-example/http-service-oauth2/>
- <https://ballerina.io/learn/by-example/http-client-oauth2-client-credentials-grant-type/>
- <https://ballerina.io/learn/by-example/nats-basic-pub/>
- <https://ballerina.io/learn/by-example/nats-basic-sub/>
- <https://lib.ballerina.io/ballerina/openapi/latest>

For more information visit our [website](#) | [documentation](#)

You can reach out to us on [Discord](#) with any questions.